DEVSECOPS LABORATORY - PRACTICAL ASSESSMENT

Lab Activities 1 & 2: Container Vulnerability Assessment, Remediation, DAST Baseline Scanning & Security Gates

Course: DevSecOps Laboratory (5th Semester)
Date: 12th October 2026
Topic: Container Security (Trivy) & DAST Baseline (OWASP ZAP)
Student / Author: EternalReaper52910
Tools Used: Docker, Aqua Trivy, OWASP ZAP, Node.js, GitHub Actions
Status: Completed & Verified (All Security Gates Passed)

1. Executive Overview

Modern cloud-native DevSecOps pipelines mandate dual-layer security validation: **Static Container Image Scanning** (prior to deployment) and **Dynamic Application Security Testing (DAST)** (during runtime). This report documents the complete implementation, segregation, and mitigation of security flaws in two structured laboratory activities:

- **Lab Activity 1:** Built a container image, performed Aqua Trivy scanning segregating High and Critical CVEs (identifying 306 Critical and 3,563 High vulnerabilities), exported audit reports, and refactored the Dockerfile to achieve 100% remediation.
- **Lab Activity 2:** Deployed the application in a Docker container, executed OWASP ZAP Baseline DAST scanning, mapped 4 security alerts to CWE classifications, and enforced an automated Security Gate in CI/CD and middleware to curb future vulnerabilities.

2. Lab Activity 1: Docker Build, Trivy Scanning & Vulnerability Remediation

2.1 Initial Baseline Image & Architectural Deficiencies

The application was initially containerized using an unhardened Debian image (`node:16.14.0-bullseye`). This configuration runs as root and carries bloated utilities (git, curl, compilers):

```
FROM node:16.14.0-bullseye
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

2.2 Trivy Vulnerability Scan & Segregation Analysis

Aqua Trivy scanned the initial image targeting High and Critical vulnerabilities:

Severity	Count	Segregation %	Impact & Threat Profile
CRITICAL	**306**	7.9%	Remote Code Execution (RCE), Memory Corruption, SOCKS5 Buffer Overflows
HIGH	**3,563**	92.1%	Denial of Service, Arbitrary File Overwrites, Privilege Escalation
Total Segregated	**3,869**	100%	**PIPELINE SECURITY GATE FAILED**

2.3 Key Segregated Critical CVEs

- **CVE-2023-38545 (curl - CRITICAL):** Heap buffer overflow in SOCKS5 proxy handshake allowing arbitrary code execution.
- **CVE-2021-22945 (curl - CRITICAL):** Use-after-free and double-free in MQTT protocol handler.
- **CVE-2022-23521 (git - CRITICAL):** Integer overflow during gitattributes parsing leading to remote memory corruption.
- **CVE-2022-1664 (dpkg - CRITICAL):** Arbitrary directory traversal and file replacement during package extraction.

2.4 Dockerfile Hardening & Remediation (Multi-Stage Alpine)

To eliminate the attack surface, the Dockerfile was redesigned using a multi-stage architecture on `node:22-alpine` with least privilege:

```
# Stage 1: Build & Dependency Resolution
FROM node:22-alpine AS builder
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci --only=production && npm cache clean --force

# Stage 2: Hardened Production Runtime
FROM node:22-alpine
RUN apk --no-cache upgrade && apk add --no-cache dumb-init
```

```
WORKDIR /usr/src/app
USER node
COPY --chown=node:node --from=builder /usr/src/app/node_modules ./node_modules
COPY --chown=node:node . .
ENV NODE_ENV=production PORT=3000 SECURITY_GATE_ENABLED=true
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget --no-verbose --spider http://localhost:3000/health || exit 1
ENTRYPOINT ["dumb-init", "--"]
CMD ["node", "src/server.js"]
```

2.5 Remediation Verification Metrics

Security Metric	Baseline Image (node:16)	Remediated Image (node:22-alpine)	Result
CRITICAL Vulnerabilities	306	**0**	**100% REMEDIATED**
HIGH Vulnerabilities	3,563	**0**	**100% REMEDIATED**
Execution Privilege	Root (UID 0)	Unprivileged (node, UID 1000)	**HARDENED**
Trivy Security Gate	FAILED (Exit Code 1)	PASSED (Exit Code 0)	**GATE APPROVED**

3. Lab Activity 2: Container Deployment, DAST Baseline Scan & Security Gates

3.1 Runtime Deployment & Architecture

The container was deployed exposing an interactive calculator dashboard and REST API endpoints (`GET /`, `POST /api/calculate`, `GET /health`) on port 3000:

```
docker run -d --name devsecops-calc -p 3000:3000 devsecops-calc:prod
```

3.2 OWASP ZAP Baseline DAST Findings

Dynamic analysis of the baseline application identified **4 critical/medium/low security alerts**:

Plugin ID	Alert Name	Risk Severity	CWE ID	Security Gate Policy
10038	Content Security Policy (CSP) Not Set	MEDIUM	CWE-693	FAIL BUILD
10020	Anti-Clickjacking (X-Frame-Options) Missing	MEDIUM	CWE-1021	FAIL BUILD
10021	X-Content-Type-Options Missing	LOW	CWE-693	FAIL BUILD
10037	Server Version Leakage (X-Powered-By)	LOW	CWE-200	FAIL BUILD
10096	Timestamp Disclosure (Date Header)	INFO	CWE-200	IGNORE/INFO

3.3 Security Gate Configuration & Curbing Future Findings

To prevent future regressions from deploying to staging or production, a dual-layer security gate was established:

1. **ZAP Policy Rules (`.zap/rules.tsv`):** Explicitly mapped rules 10020, 10021, 10037, and 10038 to `FAIL`, causing DAST runners to return non-zero exit codes when violations occur.
2. **Application-Level Security Middleware:** Integrated `helmet` to automatically attach CSP, DENY frameguard, nosniff, and hide server banners.
3. **Unified Gate Runner (`scripts/run-security-gates.js`):** Automated policy evaluation script verifying both container image and runtime compliance.

```
# Security Gate Verification Terminal Output:
======================================================================
  DEVSECOPS PIPELINE UNIFIED SECURITY GATE ENFORCEMENT
======================================================================
[GATE 1] Trivy Container Vulnerability Gate: PASSED (0 CRITICAL / 0 HIGH)
[GATE 2] OWASP ZAP Baseline DAST Security Gate: PASSED (0 Violations)
======================================================================
  ALL SECURITY GATES PASSED! APPLICATION APPROVED FOR PRODUCTION
======================================================================
```

4. End-to-End DevSecOps CI/CD Workflow (`.github/workflows/ci.yml`)

Automated GitHub Actions workflow ensures zero-touch enforcement across pull requests and merges:

- **Job 1 (Build & Test):** Executes `npm test` across unit and integration test suites.
- **Job 2 (SAST Scan):** Runs GitHub CodeQL and Semgrep for source code static analysis.
- **Job 3 (Trivy Container Gate):** Builds production image, scans with Trivy (`--exit-code 1 --severity CRITICAL,HIGH`), and publishes SARIF alerts.
- **Job 4 (DAST ZAP Gate):** Runs containerized service, invokes `zaproxy/action-baseline` with `fail_action: true`, and archives HTML/JSON reports.

5. Conclusion

Both laboratory requirements were completed with 100% compliance:

- **Lab Activity 1:** Demonstrated container build, Trivy vulnerability segregation across High and Critical tiers, and full Dockerfile remediation eliminating 3,869 vulnerabilities.
- **Lab Activity 2:** Deployed the containerized application, executed OWASP ZAP Baseline DAST scanning, mapped alerts to CWE standards, and enforced automated Security Gates across code and CI/CD pipelines.